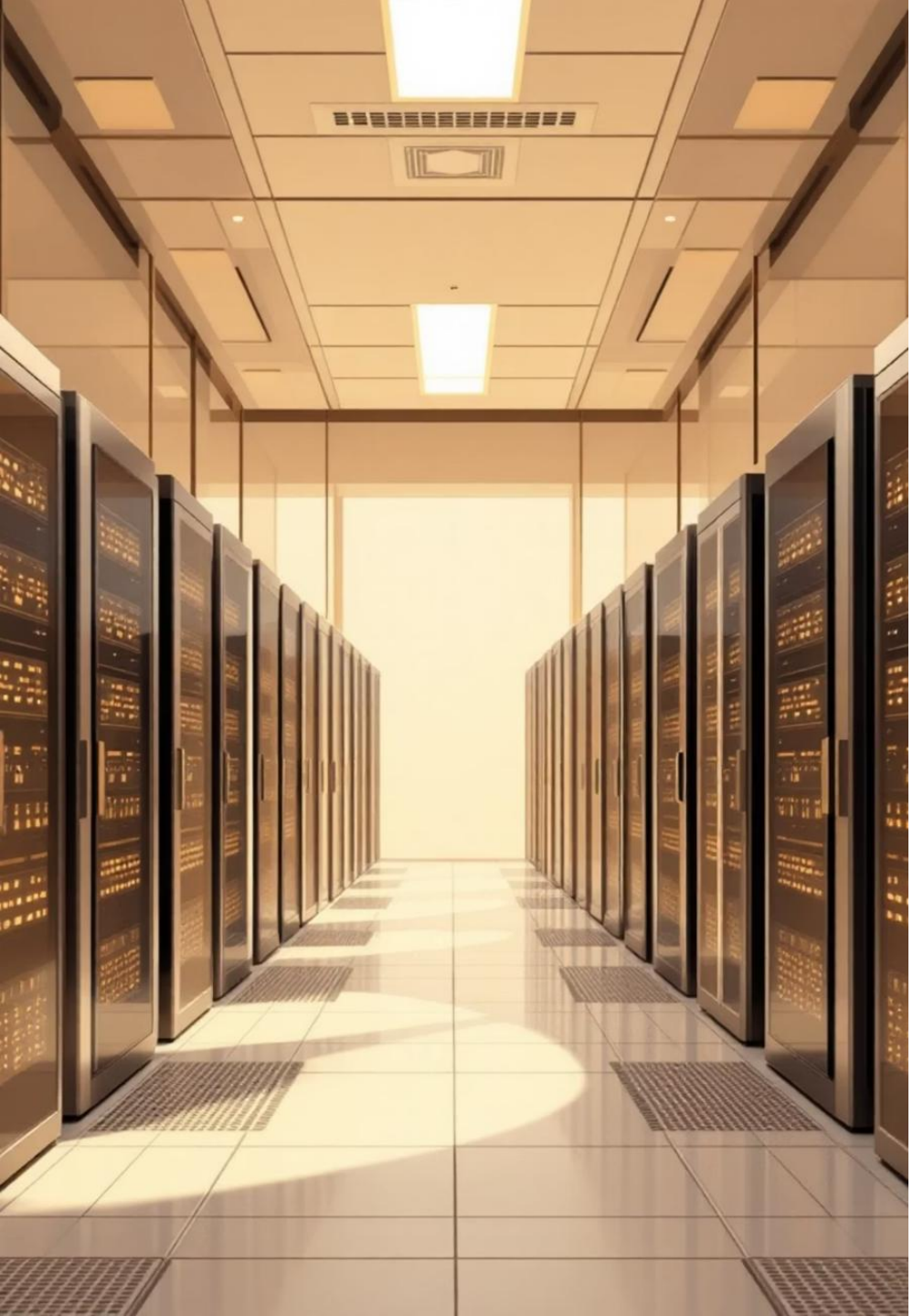
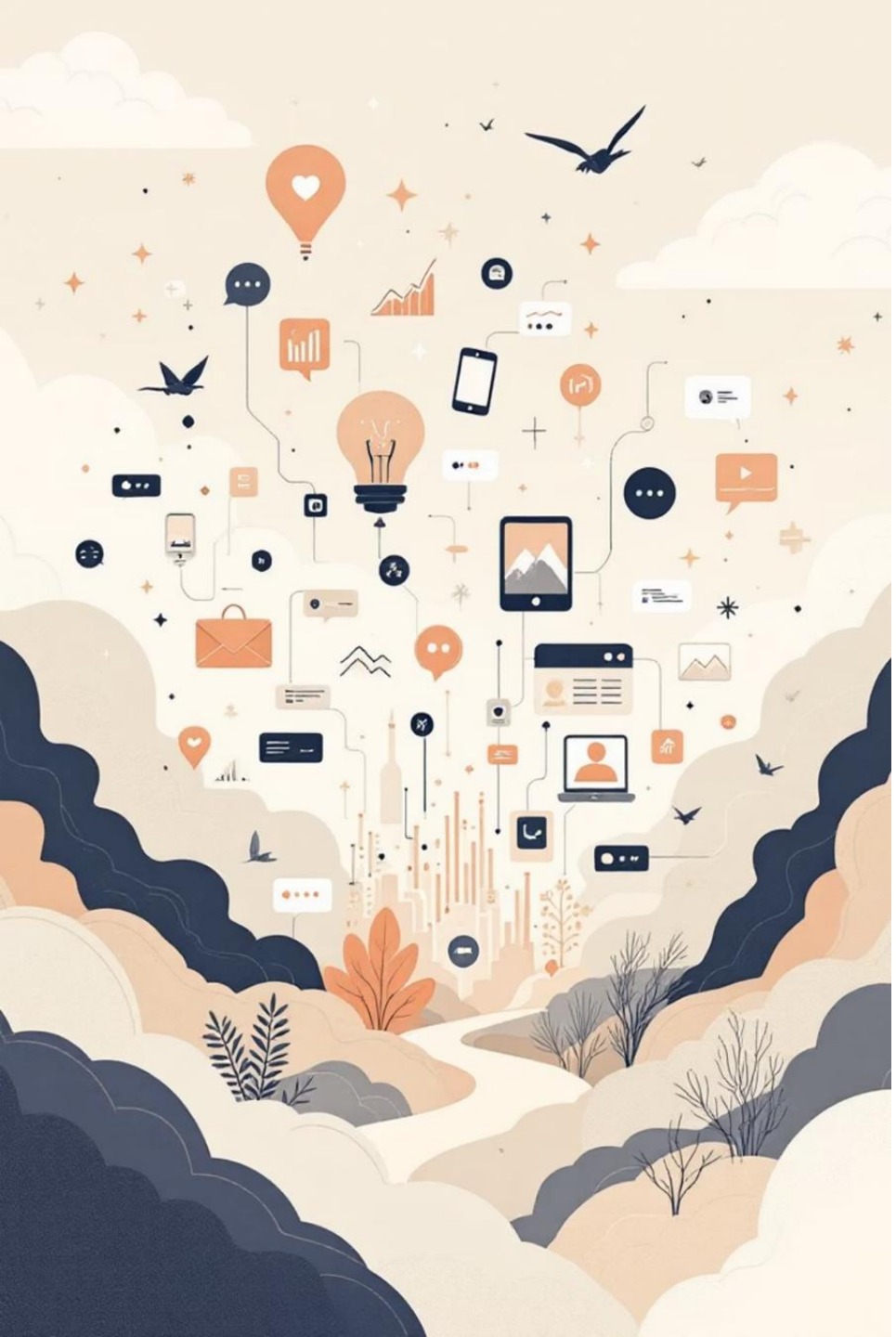


NoSQL คืออะไร? — ฐานข้อมูลยุคใหม่ที่ไม่จำกัดแค่ SQL

Woottikarn Hongwiengchan

ความหมายของ NoSQL

- NoSQL ย่อมาจาก **Not Only SQL**
- หมายถึงฐานข้อมูลที่ **ไม่ใช้โครงสร้างแบบตาราง (Table)**
- สามารถจัดเก็บข้อมูลในรูปแบบ **ยืดหยุ่น (Flexible Structure)** เช่น JSON, Key-Value, Document, Graph
- เหมาะสำหรับ **Big Data, Web App, IoT** และระบบขนาดใหญ่



เหตุผลที่เกิด NoSQL

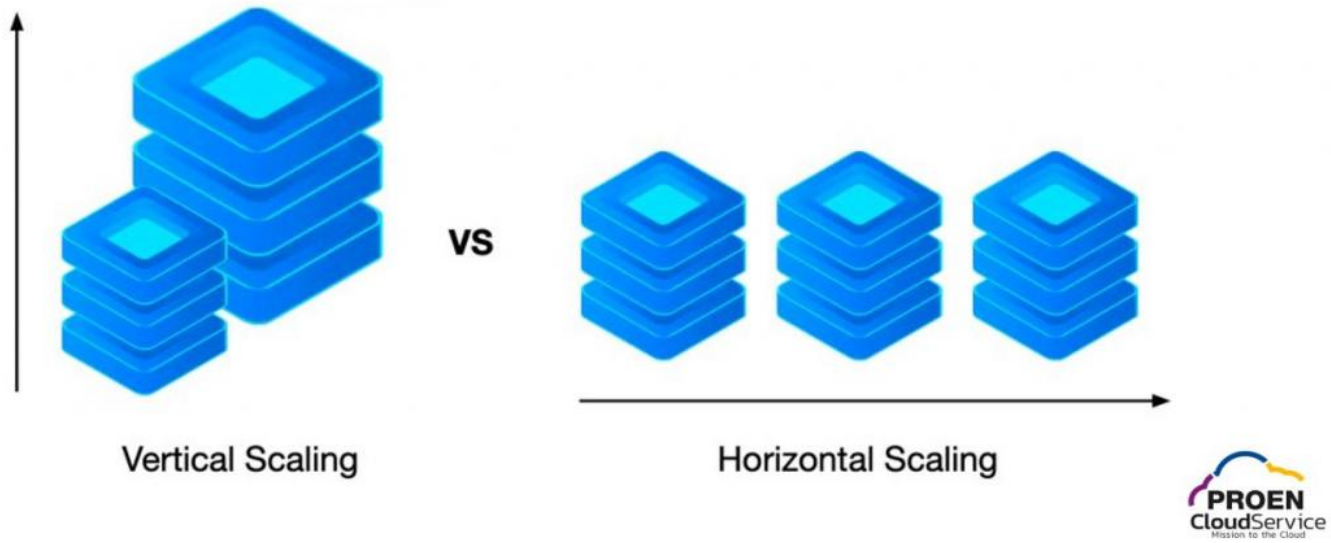
- ปริมาณข้อมูลมหาศาลจาก Social Media, IoT, E-Commerce
- ข้อมูลเปลี่ยนแปลงบ่อย มีหลายรูปแบบ (Text, Image, Sensor)
- ต้องการระบบที่ อ่าน–เขียนเร็ว และขยายระบบได้ง่าย

เปรียบเทียบ SQL vs NoSQL

รายการ	SQL (ฐานข้อมูลเชิงสัมพันธ์)	NoSQL
โครงสร้าง	ตาราง (Rows/Columns)	ยืดหยุ่น เช่น JSON
การใช้คำสั่ง	SQL Language	แล้วแต่ระบบ เช่น JSON Query
การจัดเก็บ	ต้องกำหนด Schema ล่วงหน้า	ไม่ต้องกำหนด Schema
การขยายระบบ	Vertical Scaling	Horizontal Scaling
ตัวอย่างระบบ	MySQL, PostgreSQL	MongoDB, Cassandra

การขยายระบบ (Scalability)

หมายถึงความสามารถของระบบในการ เพิ่มประสิทธิภาพหรือความสามารถในการรองรับงานเมื่อมีจำนวนผู้ใช้หรือข้อมูลเพิ่มขึ้นโดยหลัก ๆ การขยายระบบมีอยู่ 2 แบบคือ Vertical Scaling (ขยายแนวตั้ง) และ Horizontal Scaling (ขยายแนวนอน)



Vertical Scaling (การขยายแนวตั้ง)

เพิ่มขนาดหรือความสามารถของ **เครื่องเดียว (Server เดิม)** เช่น

- เพิ่ม CPU, RAM, Storage



ตัวอย่าง:

- ใช้เครื่องเดียวกันแต่เปลี่ยนจาก CPU 4 Core → 16 Core , RAM 16 GB → 128 GB

ใช้ในระบบ **SQL Database เช่น MySQL, PostgreSQL, Oracle**

ข้อดี:

- ทำง่าย เพราะไม่ต้องปรับโครงสร้างระบบ
- เหมาะกับระบบขนาดเล็กถึงปานกลาง

ข้อจำกัด:

- ถึงจุดหนึ่งจะ “ ขยายต่อไม่ได้ ” (ข้อจำกัดของเครื่องเดียว)
- ราคาเครื่องเซิร์ฟเวอร์แรง ๆ สูงมาก

Horizontal Scaling (การขยายแนวนอน)

เพิ่มจำนวนเครื่อง หลายเครื่อง (หลาย Node) มาทำงานร่วมกันข้อมูลจะถูก “กระจาย” (Sharding) ไปเก็บในแต่ละเครื่อง

ตัวอย่าง: ถ้ามีข้อมูลจำนวนมาก ⟶ แบ่งข้อมูลเก็บในเครื่อง 3 เครื่อง

- Server A เก็บข้อมูลผู้ใช้ 1–1000
- Server B เก็บข้อมูลผู้ใช้ 1001–2000
- Server C เก็บข้อมูลผู้ใช้ 2001–3000



ใช้ในระบบ NoSQL เช่น MongoDB, Cassandra

ข้อดี:

- รองรับผู้ใช้และข้อมูลจำนวนมาก
- ถ้าเครื่องใดเครื่องหนึ่งล่ม ระบบยังทำงานได้ (Fault Tolerance)
- เพิ่มเครื่องใหม่ได้เรื่อย ๆ

ข้อจำกัด:

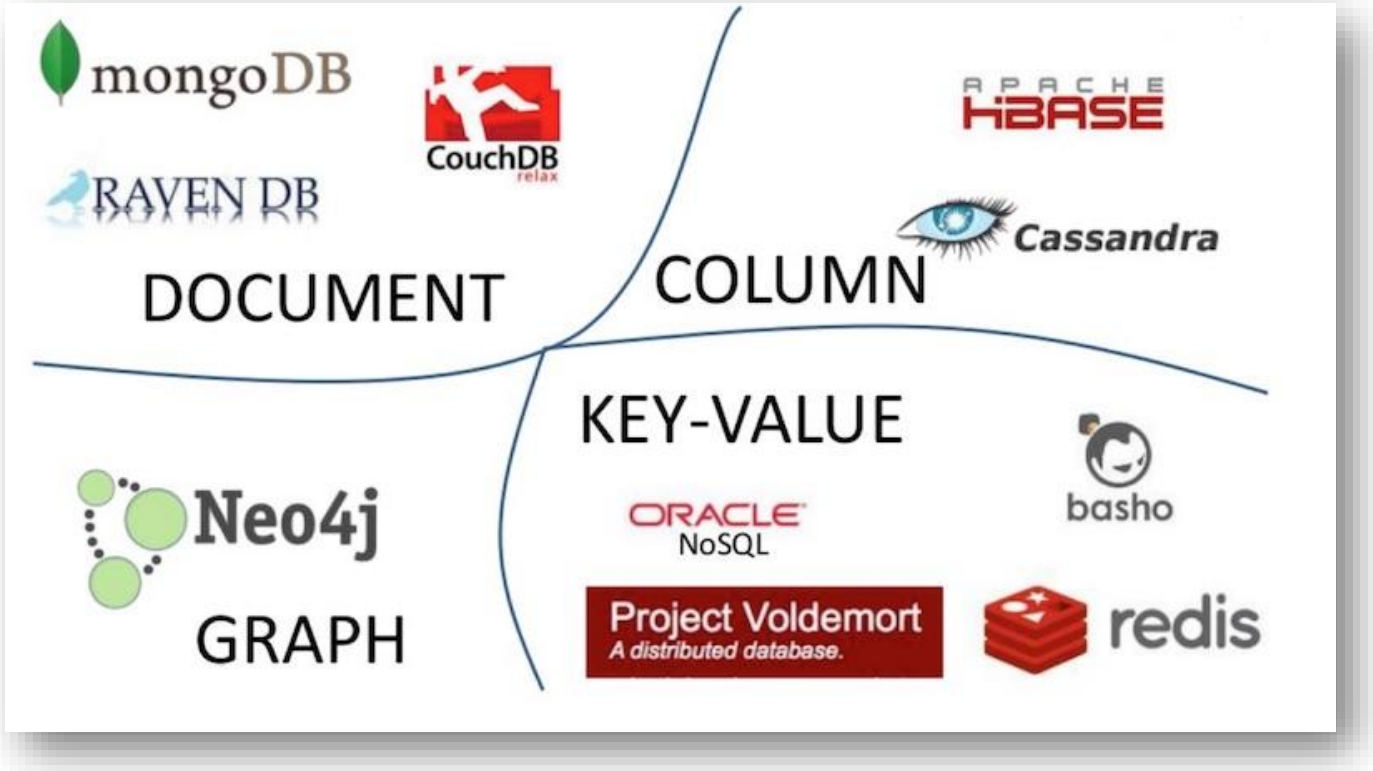
- ซับซ้อนกว่า Vertical Scaling (การขยายแนวตั้ง) ต้องจัดการระบบ Network และการ Sync ข้อมูล
- ต้องใช้เทคนิค Sharding / Load Balancing

Vertical vs Horizontal Scaling

หัวข้อ	Vertical Scaling	Horizontal Scaling
แนวคิด	เพิ่มสเปกเครื่องเดิม	เพิ่มจำนวนเครื่อง
ใช้ในระบบ	SQL	NoSQL
การจัดการข้อมูล	อยู่ในเครื่องเดียว	กระจายหลายเครื่อง
ความซับซ้อน	น้อย	มากกว่า
การขยายในอนาคต	มีขีดจำกัด	ไม่จำกัด
ตัวอย่าง	MySQL, PostgreSQL	MongoDB, Cassandra

ประเภทของ NoSQL

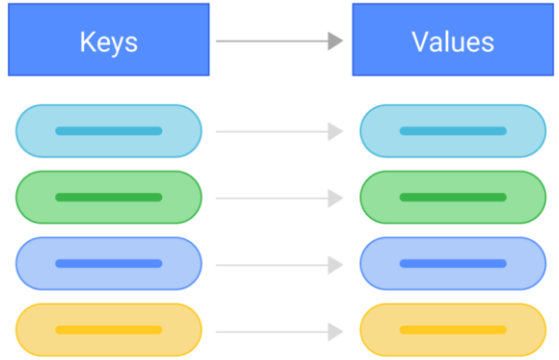
- Key-Value Store
- Document Store
- Column-Family Store
- Graph Database



Key-Value Store

จัดเก็บข้อมูลแบบคู่ Key : Value คล้าย Dictionary

ตัวอย่างระบบ: **Redis, Amazon DynamoDB**



ตัวอย่างข้อมูล:

Key	Value
user_101	{ "name": "Naruto", "village": "Konoha" }
user_102	{ "name": "Sasuke", "village": "Konoha" }

ข้อดี:

- เร็วมาก เหมาะกับการ Cache หรือเก็บ Session

ข้อเสีย:

- ค้นหาซับซ้อนไม่สะดวก

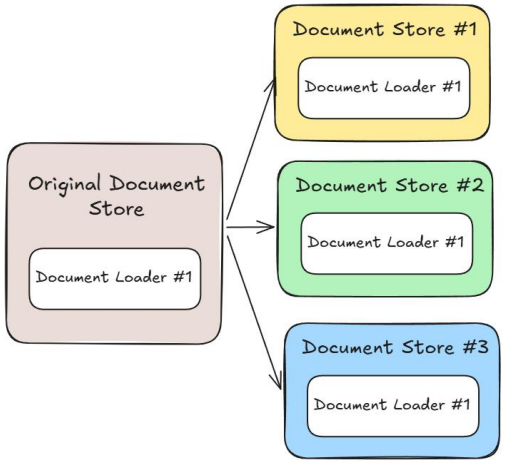
Document Store

เก็บข้อมูลเป็น “เอกสาร” เช่น JSON, BSON, XML แต่ละ Document มีโครงสร้างต่างกันได้

ตัวอย่างระบบ: MongoDB, Firebase Firestore

ตัวอย่างข้อมูล:

```
{
  "name": "Naruto Uzumaki",
  "age": 17,
  "village": "Konoha",
  "skills": ["Rasengan", "Shadow Clone"]
}
```



ข้อดี:

- ยืดหยุ่นสูง ไม่ต้องมี Schema

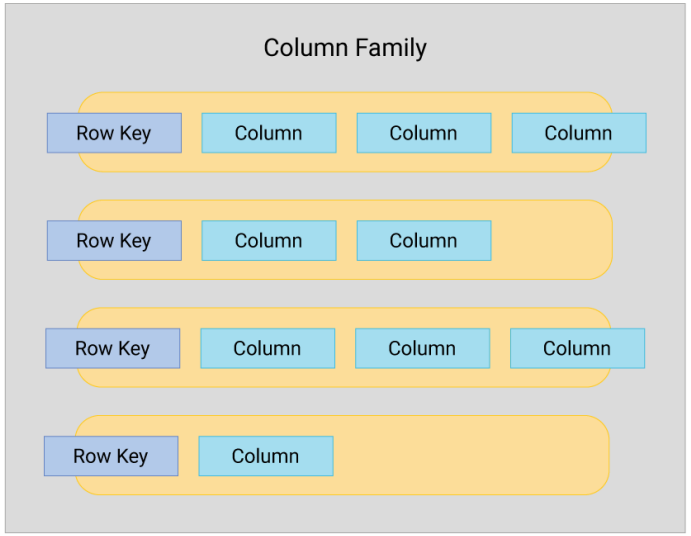
ข้อเสีย:

- ข้อมูลซ้ำได้ถ้าไม่จัดโครงสร้างดี

Column-Family Store

จะบันทึกข้อมูลในรูปแบบ Tables (Rows และ Columns) แต่จะต่างจาก RDBMS ตรงที่ แต่ละ Rows จะไม่ Fix Column (ถ้าเป็น RDBMS จะ Fix มาเป็น Pattern เดียวกัน)

ตัวอย่างระบบ: **Apache Cassandra, HBase**



ตัวอย่างข้อมูล:

UserID	Name	Email
001	Naruto	naruto@leaf.jp

UserID	Name	Phone
002	Sasuke	021-3245678

ข้อดี:

- อ่านข้อมูลเร็ว เหมาะกับ Big Data

ข้อเสีย:

- ซับซ้อนกว่า Document Store

Graph Database

เก็บข้อมูลเป็น โหนด (Node) และ เส้นเชื่อม (Edge) ใช้สำหรับข้อมูลที่มี “ความสัมพันธ์”

ตัวอย่างระบบ: **Neo4j, ArangoDB**



ตัวอย่างข้อมูล:

Node: Naruto, Sasuke
Edge: Naruto → เพื่อนกับ → Sasuke

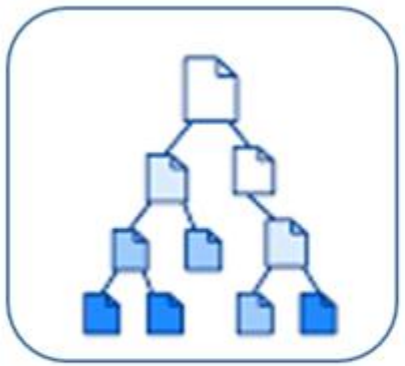
ข้อดี:

- เหมาะกับ Social Network, ระบบแนะนำ

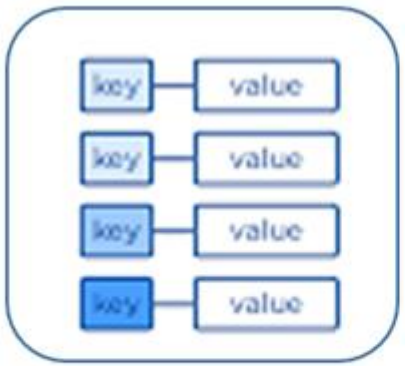
ข้อเสีย:

- Query ซับซ้อนกว่าฐานข้อมูลทั่วไป

NoSQL



Document Store



Key-Value Store



Wide-Column Store



Graph Store

ตัวอย่างระบบ NoSQL ยอดนิยม

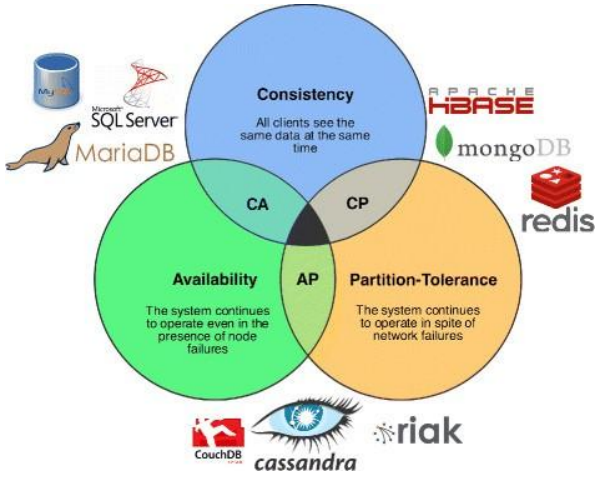
ประเภท	ระบบยอดนิยม	ใช้ในองค์กร
Key-Value	Redis	Netflix, Twitter
Document	MongoDB	Facebook, eBay
Column	Cassandra	Instagram
Graph	Neo4j	LinkedIn, Facebook

CAP Theorem

เป็นหลักการที่คิดค้นเพื่อนำมาใช้สำหรับระบบการจัดเก็บฐานข้อมูลเพื่อเป็นการการันตีว่าระบบฐานข้อมูลจะมีคุณสมบัติได้เพียง 2 ใน 3 ส่วน ได้แก่ C, A และ P

C – Consistency หมายถึง ทุกโหนดในระบบเห็นข้อมูลเดียวกันในเวลาเดียวกัน — เช่น ถ้าอัปเดตข้อมูลในโหนดหนึ่ง โหนดอื่นจะต้องเห็นค่าที่อัปเดตทันที
A – Availability หมายถึง ทุกคำขอ (**read/write**) จะได้รับการตอบกลับเสมอ — แม้ว่าผลลัพธ์จะอาจไม่ใช่ข้อมูลล่าสุด
P – Partition tolerance หมายถึง ระบบจะทำงานยังต่อได้ แม้ว่าจะมีหนึ่งใน **Database** ขาดการเชื่อมต่อหรือเสียหายไป

ซึ่ง **Database** ที่เป็นแบบ **RDBMS** ส่วนใหญ่จะเป็นแบบ **CA** เพราะว่าการที่ไม่ยอมให้ข้อมูลเสียหายเลยจำเป็นต้องตัด **P** ออก แต่สำหรับ **NoSQL** ไม่ได้ต้องการความแม่นยำของข้อมูลมากเท่าแต่ต้องการความเร็วในการเรียกใช้งานแทน ทำให้อาจจะเป็นชนิดแบบ
AP -> สามารถดึงข้อมูลจาก **Database** ได้ตลอดแม้ว่า **Database** จะเสียหายไปบางส่วน
CP -> ต้องรอให้ข้อมูล **Update** ล่าสุดก่อนถึงจะอ่านข้อมูลได้ ฉะนั้นจะมีบางช่วงเวลาที่ไม่สามารถใช้งานได้ตามความต้องการ



ข้อดี-ข้อเสีย ของ NoSQL

✓ ยืดหยุ่น ไม่ต้องกำหนด Schema
✓ รองรับข้อมูลจำนวนมาก
✓ เร็วในการอ่าน–เขียน
✓ ขยายระบบได้ง่าย (Scalable)

⚠ ไม่เหมาะกับข้อมูลที่ต้องการความสัมพันธ์ซับซ้อน
⚠ ไม่มีมาตรฐานกลางเหมือน SQL
⚠ อาจต้องพัฒนา Query เองในแต่ละระบบ

สรุปความแตกต่างระหว่าง RDBMS กับ NoSQL

	RDBMS	NoSQL
ชนิด	Relational	Non-Relational
รูปแบบของชุดข้อมูล	เป็นแบบโครงสร้างที่เก็บอยู่ใน Table	ไม่เป็นโครงสร้าง เก็บในรูปแบบ JSON (Text) หรือแบบอื่น ๆ
การ Scale	Vertical (เพิ่ม Spec Server)	Horizontal (เพิ่มจำนวน Server)
Schema	เปลี่ยนแปลงไม่ได้	เปลี่ยนแปลงได้ ค่อนข้างยืดหยุ่น
ตัวอย่าง Brand ในตลาด	Oracle, MySQL, Microsoft SQL Server, PostgreSQL	MongoDB, CouchDB, Redis, DynamoDB, Cassandra, HBase, Neo4j, Neptune

https://blog.cloudhm.co.th/sql-vs-nosql/

การเลือกใช้งาน

ใช้ RDBMS ในกรณีที่	ใช้ NoSQL ในกรณีที่
Workload คงที่ คาดเดาได้ และต้องการพื้นที่ปานกลางถึงมาก	มีปริมาณ Workload มหาศาลที่ต้อง Scale ได้เยอะ
ข้อมูลมีรูปแบบที่คาดเดาได้ มีโครงสร้างชัดเจน	ข้อมูลเป็นแบบ Dynamic มีการเปลี่ยนแปลงบ่อย
ข้อมูลต้องมี Relation เชื่อมถึงกันและกัน	ข้อมูลไม่จำเป็นต้อง Relation กันแบบซับซ้อน
ข้อมูลที่เขียนต้องเป็นไปตาม Condition	ต้องการเขียนไว ๆ Condition สำคัญรองลงมา ไม่ได้เน้นมาก
ข้อมูลค่อนข้าง Complex ต้องมีการ Query และ Report ได้	ข้อมูลค่อนข้าง Simple
ต้องการให้มี User ในการควบคุม	ต้องการให้ข้อมูลกระจายให้เข้าถึงกันได้ทุกส่วนใน Environment
จะ Deploy ใส่ Hardware ขนาดใหญ่ หรือของตัวเอง (On-Premise)	จะ Deploy บน Cloud (On-Cloud)

https://blog.cloudhm.co.th/sql-vs-nosql/

แบบฝึกหัดท้ายบท

- อธิบายความแตกต่างระหว่าง SQL กับ NoSQL
- ยกตัวอย่างระบบที่ใช้งานฐานข้อมูลแบบ NoSQL มาอย่างน้อย 2 ระบบ พร้อมอธิบาย ?
- ยกตัวอย่างระบบที่ใช้งานฐานข้อมูลแบบ RDBMS มาอย่างน้อย 2 ระบบ พร้อมอธิบาย ?